

WanderForge

Interview Preparation Handbook

A complete study companion for **APM**, **PM-1** and **SDE** interviews, built entirely from one real project: an AI trip planner that verifies its own output against real road distances before saving it.

What this is	Every fact, number and story below is taken from the WanderForge repository — its migrations, its <code>src/lib</code> modules, its test suite and its git history.
Why one project	Interviewers trust depth over breadth. One system you can defend at the level of a database policy or a unit-economics number beats five you can only summarise.
How to use it	Part 1 is shared. SDE candidates work Parts 2, 4, 6. PM candidates work Parts 3, 4, 6. Do Part 5 last — it is what you will be probed on.

Stack. Next.js 16.2 (App Router) · Supabase Postgres with Row Level Security · Groq (gpt-oss) · Google Places & Routes · Vitest

Scale of the artefact. 15 SQL migrations · 495 passing tests · ~30 pure modules under `src/lib` · 10 hand-written itineraries covering 44 days and 206 stops

How to use this handbook

Three roles, three different conversations about the same system. The table below is the shortest path through the material for each.

Role	What they are testing	Work these parts
SDE	Can you reason about correctness, concurrency and cost? Do you understand the database rather than just call it? Can you explain a trade-off you made and the evidence behind it?	1, 2, 4, 5, 6
APM	Do you find the real problem? Can you define a metric that would actually move? Do you show judgment when two good options conflict? Structured thinking under an open question.	1, 3, 4, 5, 6
PM-1	Everything APM tests, plus execution: sequencing, cutting scope, unit economics, and saying no with a reason.	1, 3, 2.4–2.5, 4, 5, 6

The 60-second pitch — memorise this

You will open with this in every interview. Say it out loud until it is 55–65 seconds.

Say it like this

“Ask any language model for a five-day itinerary and it gives you something that reads beautifully and cannot physically be done — it will put you on a mountain at 09:10 and in a restaurant across the valley at 09:30, because it has no idea how long the road takes and it will never tell you it guessed.

WanderForge is an AI trip planner whose differentiator is that it checks. Before anything is saved, every consecutive pair of activities is measured against real road distance and driving time. Journeys that do not fit the gap are named, the model gets exactly one chance to fix them, and the result is kept only if it is *measurably* more achievable than what it replaced. What survives is either achievable or explicitly flagged as not.

The interesting engineering is that verification is not free — routing is billed per leg — so the checking sits behind two shared caches and a hard cap on trip length. That is what makes an accuracy feature affordable.”

Why this pitch works. It names a problem the interviewer has personally experienced, states a differentiator that is testable rather than a claim, and ends on a constraint — which invites exactly the follow-up you are best prepared for.

The three numbers to never forget

Number	What it is	Why it matters in the room
10.1 / 21.6 / 44	Chikmagaluru town to Mullayanagiri: 10.1 km straight line, 21.6 km by road, 44 minutes driving.	This single route is the origin story of the whole product. It proves the naive estimate is wrong.
1.6×	How optimistic the flat-road estimate was on that route — 35 min predicted against 56 min real.	Turns ‘AI itineraries are unrealistic’ from an opinion into a measurement.
30	MAX_TRIP_DAYS — the hard cap on trip length.	Your cost-control answer. One model call per day plus geocoding per activity; an uncapped date range is a five-figure bill from one typo.

PART 1

The System

What it does, how it is put together, and the vocabulary you need before either track.

The problem, precisely stated

Interviewers reward a problem statement that is narrow enough to be falsified.

A language model generating an itinerary is doing plausible-text generation, not planning. It has no spatial model and no clock. It produces a schedule that is *internally* consistent — the times increase, the categories vary, the prose is confident — and *externally* impossible.

Two things make this worse than an ordinary accuracy problem:

- **The failure is invisible.** A wrong itinerary looks exactly like a right one. You only discover it standing in a car park at 09:25.
- **Prompting does not fix it.** Asking a model to check its own arithmetic produces more confident output, not more correct output. This is the key insight — the fix has to be external and deterministic.

The sentence that carries the whole project

The model is good at *choosing* and bad at *verifying*. So let it choose, and verify it somewhere else with real arithmetic.

The pipeline

Six stages between a user request and a saved trip. Learn this order — it is the spine of the system-design answer.

#	Stage	What happens	Failure mode it prevents
1	Plan	The prompt carries realism rules — road speeds by terrain, parking and queue overheads, any hop over 45 min as its own entry. These shape what the model <i>chooses</i> .	Wildly implausible drafts
2	Validate	A Zod schema coerces what it can ("9am" → 09:00, "Rs.1,200" → 1200) and rejects what it cannot. All-or-nothing.	A half-written trip
3	Resolve	Every location geocoded server-side against a shared 30-day cache.	Paying twice for the same place
4	Measure	conflictChecker walks each day pair by pair — overlaps, inverted durations, journeys that do not fit.	The silent impossible plan
5	Repair	Conflicts quoted back verbatim with a compact digest. Result kept only if measurably more achievable.	A 'fix' that is worse
6	Record	Survivors stored on the trip and shown in the editor.	Passing off a broken plan as fine

Design principle worth quoting

Everything with real logic lives in `src/lib/` as a pure module — no browser, no database, no API key required to test it. Routes and components are thin wrappers. That is why 495 tests run in about 20 seconds with no jsdom and no mocking framework.

If asked "*how do you keep a codebase testable?*", this is your answer, and it is concrete rather than aspirational.

PART 2

The SDE Track

Five deep dives, each one a real decision with real evidence, followed by the questions you will actually be asked.

2.1 Deep dive: the conflict checker

This is the algorithm interview inside your own project. Know it cold.

Core loop. For each day, walk activities in order and compare each with its predecessor. Linear in activities per day — $O(n)$ per day, $O(N)$ over the trip, where N is total activities. No pairwise blow-up, because only *consecutive* pairs can conflict on a timeline.

Distance

```
haversine(lat1, lng1, lat2, lng2) # R = 6371 km
# great-circle distance, then:
distance = roadKm ?? straightKm * ROAD_FACTOR # ROAD_FACTOR = 1.3
```

Why 1.3? Roads are never straight. 1.3 is the standard detour index for ordinary road networks. It is a fallback used only when a measured road distance is unavailable.

The sinuosity correction — the interesting bit

When the real road distance *is* known, a ratio of road to straight-line above roughly 2 means switchbacks, and no flat-road average speed applies. The model reduces speed accordingly.

Method	Predicted	Error vs measured
Great-circle × 1.3 at flat speed	35 min	1.6× optimistic
Sinuosity model, road distance only	71 min	1.27× pessimistic
Google Routes, measured	56 min	— (21.6 km, 44 min driving)

The lesson to state out loud: the best model is the one that asks. The sinuosity model exists as the fallback for when routing is unavailable, and the UI reports how many legs were *measured* versus *estimated*, so the difference is never hidden from the user.

Constants and the reasoning behind each

Constant	Value	Reasoning
ROAD_FACTOR	1.3	Detour index; fallback only
TRAVEL_TOLERANCE_MIN	10 min	Below this is noise. A checker that cries wolf gets ignored entirely — the tolerance protects the credibility of the warnings that matter.
ROAD_KEY_PRECISION	3 dp (~110 m)	Cache key rounding. Coarser = more hits but wrong legs collide; finer = correct but never hits.
SPEED_KMH	walking 4, bike 12, ...	Already discounted for real conditions, not theoretical maxima

Category exemptions — and why each exists

Two categories are exempt from the travel-time check, for two *different* reasons. Being able to separate them is exactly the precision an interviewer is listening for.

Category	Exempt because	Scope of the exemption
transport	The entry <i>is</i> the journey. The prompt asks for any hop over 45 min as its own entry; measuring the gap <i>before</i> that entry and demanding it cover the drive flagged correct plans as broken.	Both directions — the leg in and the leg out

Category	Exempt because	Scope of the exemption
accommodation	Arriving late has no consequence. The bed does not close, nothing is missed, nobody waits. But it was marking whole days unachievable.	Arrival only. Leaving late still makes you late for what follows

A strong thing to volunteer

"When I suppressed the accommodation warning I deliberately kept three things: the distance still counts toward the day's total, the long-hop warning still fires because a 120 km drive to the hotel is worth knowing about, and overlaps are untouched because two things at once is a data problem either way. Narrow the exemption to exactly the claim you are making."

2.2 Deep dive: Row Level Security

The richest database material in the project. Three separate bugs, three different classes.

Bug 1 — infinite recursion in policy evaluation

The original policies were cross-table subqueries: a policy on trips queried trip_collaborators, whose own policy queried trips. Postgres recursed until it gave up.

Fix (migration 003): move the lookups into SECURITY DEFINER helper functions. A definer function runs as its owner, so RLS on the tables it touches is not re-evaluated — the cycle is broken.

Bug 2 — privilege escalation through a missing WITH CHECK

This is the one to lead with. It is a genuine security bug and the explanation is crisp.

```
-- USING decides which rows you may SEE / target
-- WITH CHECK decides what the row may LOOK LIKE afterwards

CREATE POLICY ... FOR UPDATE
  USING (user_id = auth.uid());      -- <-- no WITH CHECK
```

With only USING, you may update a row you own — and set user_id to somebody else on the way out. You can hand your row to another account, or in the collaborator table promote yourself. Migration 007 adds explicit WITH CHECK to every UPDATE/ALL policy, plus triggers pinning immutable columns, and then **fails the migration** if any UPDATE-capable policy still lacks one.

Why the self-check matters more than the fix

Anyone can patch a policy. Ending the migration with a query over pg_policies that raises an exception if a gap remains means the class of bug cannot silently return. Say this — it reads as engineering maturity rather than bug-fixing.

Bug 3 — RLS as a performance problem

Anti-pattern	Why it is slow	The fix
id IN (SELECT ...)	Re-runs the subquery per row	Correlated EXISTS
auth.uid() bare	Re-parses the JWT for every row	(SELECT auth.uid()) — forces an InitPlan, evaluated once

Be honest about this one: migration 012 is unbenchmarked. A seed-and-measure script exists and nobody has run it. Saying so is a strength — see Part 5.

2.3 Deep dive: transactions and atomicity

The cleanest 'I understood the platform' story in the project.

The constraint. PostgREST wraps each HTTP request in its own transaction. So two requests are two transactions no matter how carefully the client sequences them. This cannot be fixed client-side — which is the whole point.

Operation	What went wrong as two requests	Consequence
Replan a day	DELETE the day, then INSERT the new one row at a time	A model timeout between the two destroyed the day with nothing to put back
Create a trip	INSERT the trip, then INSERT its days	A failure on the second left a trip the editor cannot open — no days means no timeline

Fix. Move each pair into a Postgres function so it is one transaction. Two details worth volunteering:

- **They are SECURITY INVOKER, deliberately.** RLS still applies to the tables they touch, so the itinerary lock and the editor checks are still enforced. A SECURITY DEFINER function here would quietly become a way to write into anybody's trip.
- `replace_day_activities` **refuses an empty replacement.** Every caller is replacing a day with a new plan; an empty array means the plan failed to arrive, and wiping the day on the strength of that is the exact loss the function exists to prevent.
- **Invariants are enforced, not assumed.** `create_trip_from_template` refuses a template whose day count disagrees with its declared duration, rather than shipping a trip with a phantom final day.

Interview framing

“The bug was not in my code — it was in my assumption that sequencing two calls carefully was the same as making them atomic. Once I understood that PostgREST gives one transaction per request, the fix moved from the client to the database, where it was expressible.”

2.4 Deep dive: caching, and why it is a product decision

PM-1 candidates: this section is yours too. It is where engineering meets unit economics.

Verification costs money. Every leg measured is a billed Routes call; every place resolved is a billed Places call. A five-day itinerary is roughly 35 legs, and it would be re-billed on every regeneration, every replan, and every time somebody opened the trip.

Cache	Keyed on	TTL	Why it is shareable
geocode	Normalised query + coarse bias cell	30 days	A place's coordinates are public geography. They do not differ per user.
route	From/to rounded to ~110 m + travel mode	30 days	A road between two points is public geography and rarely changes.

Three design details worth stating

- **The TTL is a licence term, not a guess.** 30 days is the Google Maps Platform caching limit. Naming that turns an arbitrary constant into a compliance decision.
- **Readers filter on `expires_at`, they do not trust the purge job.** The RLS SELECT policy itself excludes expired rows, so correctness does not depend on a cron job having run.
- **Writes are service-role only.** Readable by any signed-in user because sharing is the entire point; writable by nobody, because a user who could write travel times that everyone else's itinerary is validated against could make an impossible day look fine.

The security answer hidden in a cache

That last bullet is a genuine threat-model answer. A shared cache is a shared trust boundary: poisoning it is an attack on every other user's correctness, not just your own.

Live numbers you can quote: 33 geocode entries and 15 route entries with zero confirmed misses. Small, but it is real measurement rather than a claim.

2.5 Deep dive: streaming, and a trade-off with numbers

The single best 'show me a trade-off you measured' story in the project.

Constraint. Groq counts prompt plus reserved completion against one 8,000 token/minute allowance. That is why generation is a single completion read incrementally rather than one request per day, and why a retry carries a *digest* of the plan rather than the plan itself.

The trade-off. `response_format: json_object` guarantees parseable output — and buffers the entire completion before sending a byte.

Mode	First byte	Day one on screen	Guarantee
JSON mode on	6.4 s	~6.4 s (all 12 KB lands in 3 ms)	Always valid JSON
JSON mode off	116 ms	1.9 s	Must recover the object from raw text

Resolution: both paths exist. The streaming path drops JSON mode and uses a tolerant incremental parser that pulls whole days out of text still arriving; the non-streaming path keeps the guarantee. The parser is tested at chunk sizes from one byte upward.

Why this answer scores

It has a measured number on both sides, a decision, and a mitigation for the risk the decision introduced. That is the full shape of a trade-off answer — most candidates give only the first half.

2.6 Security and secret handling

Concern	How it is handled
User-supplied API keys	AES-256-GCM at rest with a versioned envelope. Migration 013 deletes any pre-existing plaintext key rather than attempting to encrypt it in place — a key that has been stored in the clear must be rotated, not rescued.
Column-level exposure	<code>encrypted_key</code> is revoked from client roles entirely; only a server role may write it. A <code>key_hint</code> column holds the last four characters for display.
Provider keys	Server-side only. Never <code>NEXT_PUBLIC_</code> , because that prefix ships them to the browser.
Anonymous access	Templates are readable by anon on purpose — a plan nobody can see until they sign up cannot persuade anybody to sign up. Creating a trip still requires an account, because a trip needs an owner.

Note the pattern: in each row the decision is defended by a consequence, not a convention. That is how to talk about security without sounding like a checklist.

2.7 SDE questions you should expect

“Walk me through this system.”

Use the six-stage pipeline from Part 1. Spend most of your time on stages 4 and 5 — that is the differentiator. Finish by naming the constraint that shaped it (routing is billed per leg), which sets up the caching follow-up you want.

“How would you scale this to 100,000 users?”

Answer in cost order, not traffic order — this system is bounded by spend before it is bounded by CPU. (1) The caches already make the dominant cost sub-linear: popular destinations converge to near-zero marginal geocoding. (2) Model calls do *not* amortise — they are per-trip, so that is the real unit cost; batching and a cheaper model for simple trips are the levers. (3) Only then the usual: connection pooling, read replicas, and moving generation to a queue so a slow model does not hold a request open.

Then volunteer the honest gap: the RLS performance work is unbenchmarked, and RLS is evaluated per row, so it is the most likely first bottleneck.

“Tell me about a bug you found that was not where you thought it was.”

The strongest story in the project. While testing, the template preview page 404'd on an id that the page's own link had produced. It looked like a data bug.

The server log showed `ConnectTimeoutError` — the dev server's Node process could not reach the database at all, while the browser and curl both could. Restarting it fixed it.

The real finding was second-order: the page returned `null` on *any* failure and rendered 'Template not found'. So 'the database is unreachable' and 'no such template' were indistinguishable. A transient outage shows a 404 for a row that exists. The fix is to throw on a query error and reserve 404 for a genuinely absent row.

“How do you know your tests are worth anything?”

Three concrete claims. (1) The suite is fast — 495 tests in ~20 seconds — because the logic is pure, so it actually gets run. (2) It contains regression tests pinned to real incidents, not invented cases: the main conflict-checker fixture is the actual Chikmagaluru day that exposed the transport bug, with the real coordinates. (3) It caught a real defect during this work — a migration used an activity category that violated the table's CHECK constraint, which would have failed the entire migration on execution.

“What would you refactor first?”

The error/absence conflation described above, because it actively misleads during an incident. Then the `set-state-in-effect` lint error in the navbar — harmless today, but it means `lint` is never clean, and a permanently red signal trains everyone to ignore the signal.

2.8 Where the DSA topics actually live here

Classic topic	Where it appears	Likely follow-up
Haversine / geospatial	Distance between consecutive activities	Why not Euclidean? Curvature; and lat/long degrees are not equal distances
Interval overlap	Overlap detection on a day's timeline	Sort by start, compare with predecessor — $O(n \log n)$, or $O(n)$ if already ordered
Graph / shortest path	Road distance and duration between stops	Why call a provider instead of implementing Dijkstra? Road networks need live data you do not have
Hashing / cache keys	Rounded-coordinate keys in both caches	Precision as a hit-rate vs correctness trade-off — $3 \text{ dp} \approx 110 \text{ m}$
Greedy / optimisation	Debt simplification in expense settlement	Minimising transfers; integer cents so balances reconcile exactly
Streaming parsing	Extracting whole days from incomplete JSON	State machine over brace depth; tested from one-byte chunks upward

PART 3

The PM Track

Product thesis, metrics, unit economics and four real prioritisation calls with the reasoning attached.

3.1 The product thesis

Element	The answer
Problem	AI trip planners produce itineraries that read well and cannot be executed. The user does not discover this while planning — they discover it while travelling, when it is expensive.
Insight	The failure is not that the model is bad at travel. It is that generation and verification are different tasks, and asking one system to do both produces confidence rather than correctness.
Solution	Separate them. Let the model choose; verify deterministically against real road data before anything is saved.
Differentiator	Falsifiable. 'Better itineraries' is a claim; 'this journey is 21.6 km and 44 minutes and your plan allows 20' is arithmetic.
Wedge	Trips where the naive estimate is most wrong — mountain and rural routes. That is precisely where every competitor is worst and where the error is largest.

The positioning line

Every competitor is racing to generate itineraries faster. The differentiator is not speed of generation — it is that ours is the only one that will tell you when it is wrong.

3.2 Who it is for

Resist the urge to say 'travellers'. Segment by where the pain is largest.

Segment	Job to be done	Why the verification matters to <i>them</i>
Self-drive road-trippers	"Plan a route I can actually drive in the days I have."	Highest error segment. Winding roads are where the naive estimate fails by 1.6× or more. This is the beachhead.
Group trip organisers	"Agree a plan with five people and stop re-litigating it."	Collaboration, the itinerary lock and expense settlement. They feel the pain of a plan that changes.
First-time visitors	"Don't let me plan something stupid in a city I don't know."	No local intuition to catch an impossible hop, so they are the least able to self-correct.

3.3 Metrics

The most common APM failure is a north star that cannot move or cannot be measured. Pick one that captures the differentiator.

Tier	Metric	Why this one
North star	Trips that reach the destination date without being abandoned	Captures real value — a plan good enough to actually travel on. Not 'itineraries generated', which rewards regeneration caused by <i>bad</i> output.
Input	Share of generated trips with zero hard conflicts on first pass	Directly measures the differentiator, and is a clean A/B target for prompt changes
	Repair success rate — how often the one retry produces a measurably better plan	Tells you whether stage 5 earns its cost
	Measured-leg coverage — legs routed vs estimated	Quality of the verification itself; a low number means you are trusting the fallback
Guardrail	Cost per completed trip	The feature is billed per leg. Quality that scales the bill linearly is not a feature, it is a liability

Tier	Metric	Why this one
	Time to first day on screen	Already measured at 1.9 s. Protects against a 'quality' change that destroys perceived speed
	False-positive conflict rate	A checker that cries wolf gets ignored, which silently destroys the entire value proposition

The guardrail that shows judgment

Volunteering the false-positive rate is the strongest metric answer available here, because it recognises that a verification product fails *most* dangerously when it is over-eager. The 10-minute tolerance and both category exemptions exist to protect exactly this number — you have already shipped changes in service of it.

3.4 Unit economics

PM-1 interviews probe this. Most candidates cannot cost their own feature; you can.

Cost centre	Scales with	Control in place
Model generation	Days per trip — one completion, sized by day count	MAX_TRIP_DAYS = 30, enforced client <i>and</i> server side
Geocoding	Activities per trip	Shared 30-day cache; popular destinations converge to ~0
Route measurement	Legs per trip (~35 on a five-day plan)	Shared 30-day cache; re-checks and reopens are free

The three things to say about this table

- **Two of three costs amortise across users; one does not.** Geocoding and routing are public geography and get cheaper as the user base concentrates on popular destinations. Model generation is per-trip and does not. So the model call is the true marginal cost and the only one worth optimising with a cheaper tier.
- **The cap is a product decision disguised as a constant.** Uncapped, a mistyped end year is one model call per day plus geocoding per activity — a five-figure bill from a single typo. It is enforced server-side because the day count arrives from the client and cannot be trusted.
- **Caching converts a variable cost into a fixed one.** That is what makes verification viable as a free-tier feature at all — without it, accuracy would be a premium SKU.

3.5 Four real prioritisation calls

Each of these actually happened. Rehearse them as decisions with a reason, not as anecdotes.

Call 1 — 30 shallow templates, or 10 real ones?

Option	Case for	Case against
30 destinations, no plans	Looks abundant; wider search coverage	'Template' promises a plan; what arrived was an empty calendar. The word wrote a cheque the product did not honour.
10 with full plans	Each one is immediately usable and screenshot-worthy	Two-thirds less catalogue

Decision: ten. **Reasoning:** thirty stubs that say nothing are worth less than ten plans somebody can follow. The catalogue was optimising a vanity number; the plans optimise activation. **Say this:** "I cut the catalogue by two-thirds to make the feature match its own name."

Call 2 — should logged-out visitors see full itineraries?

Tension: gating the plan protects a signup; showing it gives the value away. **Decision:** show everything, require an account only to *save an editable copy*. **Reasoning:** a plan nobody can see until they sign up cannot persuade anybody to sign up. The gate moved from *seeing value* to *owning value* — which is also the point at which a database row genuinely needs an owner. Ships an SEO-indexable surface as a side effect.

Call 3 — CI and error monitoring: not merged

Both were built and deliberately left unmerged as unnecessary for a side project. That was a defensible call. **It stops being defensible the moment you demo to an investor**, because at that point ‘I think it works’ competes against ‘no errors in the last thousand sessions’. Being able to name when a reasonable decision expires is a senior-sounding answer.

Call 4 — suppressing the accommodation warning

A user reported the checker flagging late arrival at a homestay as an error. **The lazy fix** is to exempt the category. **The correct fix** was to ask what claim the warning makes: being late matters when something is missed. Nothing is missed by reaching your own bed later. So the exemption covers *arrival only* — leaving late still makes you late for what follows — and the distance still counts, and the long-hop warning still fires.

The transferable principle

Narrow an exemption to exactly the claim being made. A category-wide suppression would also have hidden a genuinely broken plan that happened to end at a hotel.

3.6 The case study to lead with: silent failure

If you get one ‘tell me about a product bug’ question, use this. It is about a class of failure PMs are uniquely responsible for catching.

What happened	Choosing a destination for its food and culture sent the planner <i>nothing</i> . The template flow wrote the user's choices to <code>ai_preferences.tags</code> ; the generation route reads <code>ai_preferences.interests</code> .
Why nobody noticed	Nothing errored. A trip was created. Days were planned. The output was plausible — just generic. Every template produced roughly the same trip, and the destination's whole character was thrown away silently.
Why it is a PM bug	No test would catch it: both fields are valid JSON and every layer behaved correctly in isolation. Only somebody asking ‘did the output actually reflect what the user asked for?’ would find it.
The second failure	The two vocabularies did not even match. <i>romance</i> , <i>luxury</i> and <i>technology</i> are reasonable things to say about a city and are not interests the planner understands. Passing them through unmapped would have been ignored a <i>second</i> time.
The fix	An explicit mapping to the vocabulary the planner reads, with a test asserting that every template maps entirely into it — so nothing can reach the planner that it silently drops.

The line that lands

“The bug was not that something broke. It was that the product kept working while quietly ignoring the only input the user gave it. Those are the expensive ones, because no alert fires and no test fails — the only detection is someone checking that the output reflects the request.”

3.7 Product-sense questions, with the structure to use

“How would you improve WanderForge?”

Do not list features. Segment by funnel stage and name the evidence gap first: *you cannot currently answer where users drop off, because there is no analytics*. That is the honest starting point and it scores well.

Then propose in order: **(1) Activation** — templates now produce a complete plan in one click, so measure whether template-started trips retain better than wizard-started ones. **(2) Trust** — surface ‘N of M journeys measured’ more prominently; it is the differentiator and it is currently buried in a panel. **(3) Retention** — nothing brings a user back after the trip is planned; the obvious hook is the trip itself, day by day, while they are travelling.

“Your conflict checker flags 40% of generated trips. Ship it or not?”

A trap question about false positives. Split the number before answering: how many are *hard* conflicts (arithmetic — the journey does not fit) versus *soft* (inferred opening hours, which the system already treats as advisory)?

Hard conflicts at 40% means the prompt is wrong and should be fixed upstream. Soft warnings at 40% means the threshold is wrong and is training users to ignore all warnings, which destroys the feature. The 10-minute tolerance and both category exemptions exist precisely to hold this number down — cite them as prior art.

“What would you kill?”

Name real dead weight — it shows you audit rather than accumulate. Three installed-but-unused dependencies (charting, drag-and-drop, offline storage), an email dependency with no caller so invitations are stored but never sent, and two schema tables nothing reads.

Then make the point: the invitations one is not dead weight, it is a *broken promise* — the UI implies an email was sent. Either send it or stop implying it. Unused code is clutter; an unkept promise is a bug.

PART 4

Behavioural Stories

Six STAR answers drawn from work that actually happened, each mapped to the competency it demonstrates.

How to use these

Each story is real. Keep Situation and Task to two sentences; interviewers cut you off during setup, never during result. Lead with the result if the interviewer seems rushed.

Debugging under a false signal

Demonstrates: Analytical rigour · not fooling yourself

Situation. A navigation link was reported as doing nothing. I reproduced it and found the page would not scroll at all — `window.scrollTo` was a no-op. I had a culprit.

Task. Confirm the cause before fixing, because a fix aimed at the wrong cause is worse than no fix.

Action. Instead of fixing what I had 'found', I ran the same test on an unrelated plain page as a control. It also refused to scroll — so the failure was in my headless test environment, not the app. I threw the reproduction away and diagnosed from the source instead.

Result. The real bug was different and simpler: the links were bare fragments, and the navbar renders on every page, so from any page other than the landing page they pointed at an element that did not exist. A control experiment cost two minutes and prevented shipping a fix for a bug that was not there.

Cutting scope by two-thirds

Demonstrates: Product judgment · saying no

Situation. The explore page offered thirty destinations. 'Use template' created a trip with days and no activities — an empty calendar.

Task. Make the feature match the promise in its own name.

Action. I cut the catalogue to ten and wrote genuine day-by-day itineraries for each — 44 days and 206 stops, every stop with a time, a category, a cost and real coordinates. I argued that thirty stubs saying nothing were worth less than ten plans somebody could follow.

Result. Every template is now immediately usable and the feature is the most demonstrable thing in the product. The decision also made the catalogue honest: nothing is listed that cannot be used.

A security bug in a database policy

Demonstrates: Depth · preventing the class, not the instance

Situation. Update policies specified only which rows a user could target, with no constraint on what the row could look like afterwards.

Task. Close the hole, and make the same class of hole impossible to reintroduce.

Action. I added an explicit constraint to every update-capable policy, plus triggers pinning columns that must never change — ownership, identity. Then I ended the migration with a query over the system catalogue that raises an exception if any update policy still lacks one.

Result. The specific escalation path — handing your own row to another account, or promoting yourself in the collaborator table — was closed, and the migration now fails loudly rather than silently if the gap ever returns.

Finding a bug before it ran

Demonstrates: Care · verifying rather than assuming

Situation. I had written a database migration seeding ten itineraries with over 200 activities. It looked correct and the tests passed.

Task. Validate it before running it against a live database, since a migration is hard to take back.

Action. I wrote a throwaway script that parsed the SQL, extracted every embedded JSON block and checked it against the actual table constraints — categories, time formats, coordinate ranges, and whether each template's declared duration matched its plan.

Result. It found 24 activities using a category that the table's CHECK constraint does not allow. That single error would have failed the entire migration on execution. The lesson I keep: validate against the real constraint, not against what you believe the constraint to be.

Reversing my own prior decision

Demonstrates: Humility · responding to evidence

Situation. A user reported the conflict checker flagging late arrival at a homestay as an error. I had previously written a test asserting exactly the opposite behaviour — that accommodation should *not* be exempt.

Task. Change the behaviour without discarding the reason the original test existed.

Action. The old test was guarding a different exemption from over-broadening. Rather than delete it, I kept its intent and moved its example to a category where lateness still has consequences. Then I made the new exemption as narrow as the claim: arrival only, distance still counted, long-hop warning still firing.

Result. The false positive is gone and the original guard is intact. I would rather understand why the previous decision was made than assume it was careless.

Reporting a failure honestly

Demonstrates: Integrity · calibrated confidence

Situation. I was asked to verify a feature end to end. One page returned 'not found' for a record that demonstrably existed.

Task. Report what was actually true, not the tidiest story.

Action. I traced it to the local server being unable to reach the database, restarted it, and the page worked. I could have reported success. Instead I flagged a second-order finding: the page returned the same 'not found' for a missing record and for an unreachable database, so a transient outage would show a 404 for a record that exists — which is exactly what had just misled me.

Result. The immediate task passed, and a real defect that would only ever surface during an incident was documented. I also flagged that one critical path remained unverified rather than implying full coverage.

PART 5

Known Weaknesses

What a good interviewer will find. Say it before they do — it converts a gap into evidence of judgment.

Answering weakness questions

The pattern for every row below: **state the gap plainly, explain the decision that produced it, then name what would change your mind.** Never defend a gap; never apologise for it either.

Gap	How to answer it
No analytics	"I cannot tell you how many people used it, and that is the first thing I would fix before pitching it. It was built to prove a technical thesis, and I optimised for that. The instrumentation gap is the difference between a project and a product."
Not load-tested	"No. And I can tell you exactly where I would expect it to break first — row-level security is evaluated per row, and my performance work there is unbenchmarked. There is a seed-and-measure script and I have not run it."
No CI on main	"A deliberate call for a side project, and one that expires the moment anyone else contributes or I demo it. The pipeline exists on a branch."
Unused dependencies	"Charting, drag-and-drop and offline storage are installed and unused — features I scoped and did not build. I would rather remove them than leave the impression they exist."
Invitations never sent	"The worst one, because it is not missing — it is <i>misleading</i> . Invitations are stored and the UI implies an email went out. Either send it or stop implying it."
Legal text unreviewed	"Accurate about what the software does, and not reviewed by a lawyer. Each document says so at the top and lists what a real deployment must fill in."
A path never run	"The function that turns a template into a trip is validated structurally and has not been executed against a live database. I know precisely which line I would watch fail."

Why this section is an asset

Most candidates present a project as finished. Presenting one with a precise, unprompted account of its limits signals that you know the difference between working and shipped — which is the single most valuable instinct at APM/PM-1 and at SDE alike.

The two questions most likely to expose you

"How do you know the verification is actually correct?"

The honest answer has three layers. (1) The arithmetic is unit-tested against a real route with known ground truth — measured driving time from a routing provider. (2) When routing is unavailable, the fallback is *knowingly* imperfect, and the UI reports how many legs were measured versus estimated rather than hiding the difference. (3) What I have **not** done is validate it against a corpus of real trips at scale. One route is an anchor, not a proof.

"Isn't this just a wrapper around an LLM?"

The direct answer: the model is one of six stages, and it is the only one that is not deterministic. The schema contract, the geocoding, the arithmetic, the repair-acceptance rule and the storage of unresolved conflicts are all ordinary engineering. Remove the model and you still have a verifier; remove the verifier and you have what everyone else already ships.

PART 6

Rapid Revision

The night before. Numbers, one-line answers, and the final checklist.

Numbers to have on instant recall

Value	What it is
10.1 km / 21.6 km / 44 min	Chikmagaluru → Mullayanagiri: straight line, road, driving time
35 / 71 / 56 min	Flat estimate / sinuosity model / measured — the whole argument in three numbers
1.6×	How optimistic the naive estimate was on that route
1.3	ROAD_FACTOR — detour index, fallback only
10 min	Conflict tolerance. Below this is noise that would train users to ignore warnings
30	MAX_TRIP_DAYS — the cost cap
30 days	Both cache TTLs — a Google Maps licence term, not a guess
~110 m	Route cache key precision (3 decimal places)
8,000 tokens/min	Groq allowance covering prompt <i>plus</i> reserved completion
116 ms vs 6.4 s	First byte without / with JSON mode
1.9 s	Time to first day on screen
495 / ~20 s	Tests passing, and how long they take
10 / 44 / 206	Templates, days, individual stops

One-line answers

If asked...	Say
What is it?	An AI trip planner that measures its own output against real road distances before saving it
What is hard about it?	Verification is billed per leg, so accuracy has a unit cost — the engineering is making it affordable
Why won't prompting fix it?	A model asked to check its own arithmetic produces more confident output, not more correct output
Biggest technical insight?	PostgreSQL gives one transaction per request, so multi-step writes must move into the database
Biggest product insight?	The gate belongs at owning value, not seeing it
Biggest mistake?	Believing a reproduction before running a control
What is missing?	Analytics. I cannot tell you how many people used it

Final checklist

- Can you deliver the 60-second pitch without notes, in under 65 seconds?
- Can you draw the six-stage pipeline from memory and name what each stage prevents?
- Can you explain USING versus WITH CHECK in one sentence each?
- Can you state the streaming trade-off with the number on both sides?
- Can you name your north-star metric and, more importantly, the guardrail that protects it?
- Can you cost a single trip across all three cost centres and say which one does not amortise?
- Do you have two stories where you were *wrong* and what changed your mind?
- Can you name four real weaknesses without becoming defensive about any of them?

The last thing to remember

The strongest signal in this entire project is not the algorithm — it is that at several points you chose the more accurate answer over the more flattering one: cutting the catalogue by two-thirds, reversing your own test, reporting a failure you could have quietly fixed. Lead with the work; let that pattern show through it.